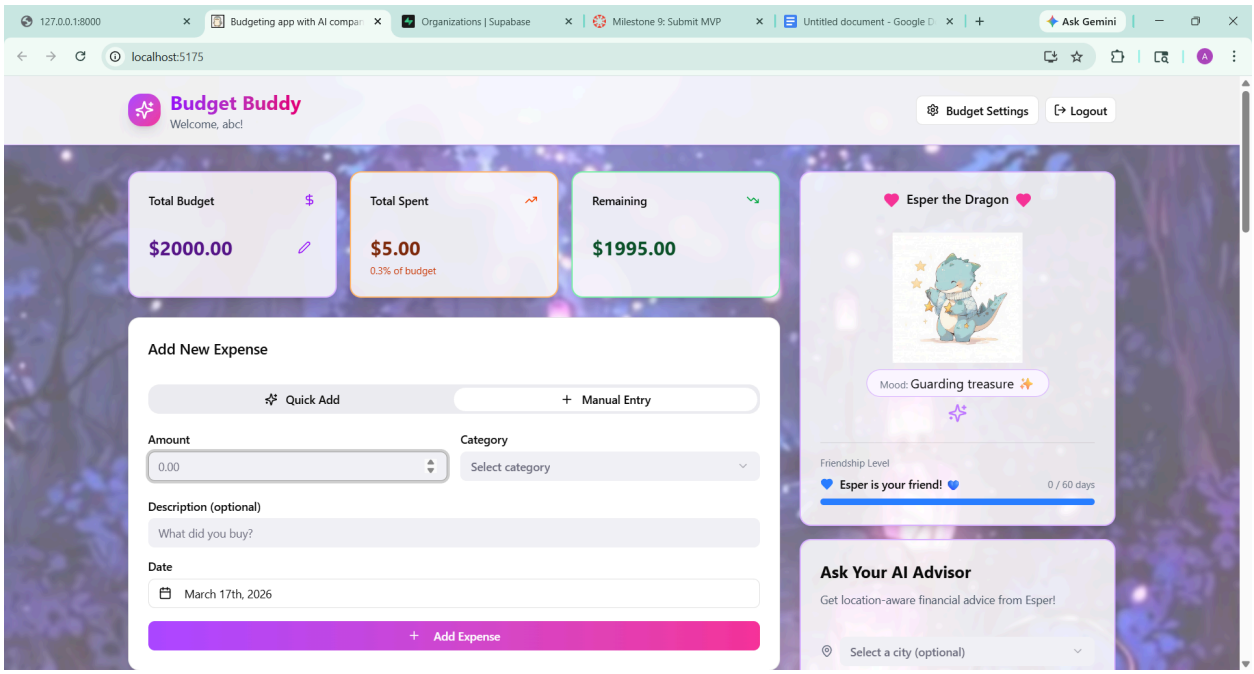
1. Deliver a **working prototype (MVP)** that includes the core features of your project.



2. Include a basic user interface (e.g., Streamlit or Gradio) or provide functional API endpoints.

Frontend UI in BudgetBuddy.tsx and related React screens.

Functional backend endpoints in main.py, including /api/chat, auth endpoints, expense endpoints, and health checks.

3. Demonstrate **end-to-end functionality**, showing how the system processes inputs and produces outputs.

Budget Buddy demonstrates end-to-end functionality from UI input to backend processing to persisted results and model output.

User authentication flow

Input: user enters a username in the login screen.

Processing: frontend sends a login request to the backend auth endpoint, backend creates or retrieves the user and returns a token.

Output: user session starts in the app, token is stored client-side for authenticated requests.

Project evidence: Login.tsx, main.py, auth.py

Expense tracking flow

Input: user submits expense amount, category, description, and date from the app UI.

Processing: frontend calls expense endpoints, backend validates payloads, writes records through database client, and recomputes summaries.

Output: updated expense list, budget totals, and visual summaries are shown in the app.

Project evidence: App.tsx, SpendingForm.tsx, main.py, database.py

Chat assistant flow

Input: user asks a finance question such as Give me a spending plan for Charlotte.

Processing: backend gathers user budget context, infers city context, retrieves relevant finance content from indexed sources, then generates a grounded response.

Output: chatbot returns a practical spending plan with cited source context instead of a mock response.

Project evidence: BudgetBuddy.tsx, main.py, llm_pipeline.py, rag.py

API reliability and health flow

Input: health check request from local host.

Processing: backend checks core services (database, llm, cost-of-living component).

Output: structured JSON health status for runtime verification.

Project evidence: main.py

This MVP therefore shows full input to processing to output across authentication, transaction management, analytics context, and assistant response generation.

4. Present a **5-minute in-class demo** of your MVP to receive peer and instructor feedback.